

01_clustering

May 9, 2026

1 Clustering B2B por dos capas

Este notebook implementa el analisis completo de clustering para la tesis usando las matrices finales ya procesadas en 03_feature_engineering/outputs.

- Capa 1: modelo principal basado en features SRI de cobertura amplia.
- Capa 2: analisis de robustez sobre el subconjunto con informacion financiera SCVS.
- es_cliente_fpa se usa solo como validacion externa y nunca entra al modelo.

Nota metodologica: la documentacion narrativa menciona tamanos de muestra 163/93, pero los CSV exportados mas recientes del pipeline contienen 175/92. Este notebook toma como fuente de verdad los artefactos actuales del repositorio.

```
[1]: # Celda 0 - Imports y rutas
from pathlib import Path
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from IPython.display import display
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, davies_bouldin_score, \
    adjusted_rand_score
from scipy.cluster.hierarchy import linkage, dendrogram

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", context="talk")

ROOT = Path("e:/TESIS MAESTRIA/Desarrollo_clustering_maestria")

PATH_FEAT_C1 = ROOT / "03_feature_engineering/outputs/features_capa1.csv"
PATH_FEAT_C2 = ROOT / "03_feature_engineering/outputs/features_capa2.csv"
PATH_MAT_C1 = ROOT / "03_feature_engineering/outputs/matriz_capa1.csv"
PATH_MAT_C2 = ROOT / "03_feature_engineering/outputs/matriz_capa2.csv"
PATH_LAB_C1 = ROOT / "03_feature_engineering/outputs/labels_capa1.csv"
PATH_LAB_C2 = ROOT / "03_feature_engineering/outputs/labels_capa2.csv"
```

```
PATH_OUT = ROOT / "04_modeling/outputs"
PATH_OUT.mkdir(parents=True, exist_ok=True)
```

```
RANDOM_STATE = 42
K_RANGE = list(range(2, 9))
```

```
print(f"ROOT: {ROOT}")
print(f"Output dir: {PATH_OUT}")
```

ROOT: e:\TESIS MAESTRIA\Desarrollo_clustering_maestria

Output dir: e:\TESIS MAESTRIA\Desarrollo_clustering_maestria\04_modeling\outputs

1.1 1. Carga de datos y control de calidad

Las matrices ya llegan escaladas y codificadas desde 03_matriz_final.ipynb. En esta seccion solo verificamos consistencia, cobertura y trazabilidad entre matrices, labels y features crudas.

```
[2]: features_c1 = pd.read_csv(PATH_FEAT_C1, dtype={"RUC": str})
features_c2 = pd.read_csv(PATH_FEAT_C2, dtype={"RUC": str})
matriz_c1 = pd.read_csv(PATH_MAT_C1)
matriz_c2 = pd.read_csv(PATH_MAT_C2)
labels_c1 = pd.read_csv(PATH_LAB_C1, dtype={"RUC": str})
labels_c2 = pd.read_csv(PATH_LAB_C2, dtype={"RUC": str})

assert len(features_c1) == len(matriz_c1) == len(labels_c1), "Desalineacion en
↳capa 1"
assert len(features_c2) == len(matriz_c2) == len(labels_c2), "Desalineacion en
↳capa 2"
assert features_c1["RUC"].equals(labels_c1["RUC"]), "RUC no alineado en capa 1"
assert features_c2["RUC"].equals(labels_c2["RUC"]), "RUC no alineado en capa 2"
assert matriz_c1.isna().sum().sum() == 0, "NaN en matriz_capa1"
assert matriz_c2.isna().sum().sum() == 0, "NaN en matriz_capa2"

resumen_capas = pd.DataFrame(
    [
        {
            "capa": "Capa 1",
            "n_documentado": 163,
            "n_csv_actual": len(matriz_c1),
            "n_features_modelo": matriz_c1.shape[1],
            "clientes_fpa": int(labels_c1["es_cliente_fpa"].sum()),
            "ruc_unicos": labels_c1["RUC"].nunique(),
            "nullos_matriz": int(matriz_c1.isna().sum().sum()),
        },
        {
            "capa": "Capa 2",
            "n_documentado": 93,
            "n_csv_actual": len(matriz_c2),
```

```

        "n_features_modelo": matriz_c2.shape[1],
        "clientes_fpa": int(labels_c2["es_cliente_fpa"].sum()),
        "ruc_unicos": labels_c2["RUC"].nunique(),
        "nulos_matriz": int(matriz_c2.isna().sum().sum()),
    },
]
)

display(resumen_capas)

```

	capa	n_documentado	n_csv_actual	n_features_modelo	clientes_fpa	\
0	Capa 1	163	167	15	53	
1	Capa 2	93	87	21	37	

	ruc_unicos	nulos_matriz
0	157	0
1	87	0

```

[3]: resumen_columnas = pd.DataFrame(
    {
        "matriz_capa1": pd.Series(matriz_c1.columns),
        "matriz_capa2": pd.Series(matriz_c2.columns),
    }
)

validaciones = pd.DataFrame(
    [
        {
            "check": "Duplicados RUC capa1",
            "valor": int(labels_c1["RUC"].duplicated().sum()),
        },
        {
            "check": "Duplicados RUC capa2",
            "valor": int(labels_c2["RUC"].duplicated().sum()),
        },
        {
            "check": "Clientes FPA capa1",
            "valor": int(labels_c1["es_cliente_fpa"].sum()),
        },
        {
            "check": "Clientes FPA capa2",
            "valor": int(labels_c2["es_cliente_fpa"].sum()),
        },
    ]
)

display(validaciones)

```

```
display(resumen_columnas)
```

	check	valor
0	Duplicados RUC capa1	10
1	Duplicados RUC capa2	0
2	Cientes FPA capa1	53
3	Cientes FPA capa2	37

	matriz_capa1	matriz_capa2
0	tipo_sociedad	tipo_sociedad
1	obligado_contabilidad	obligado_contabilidad
2	es_agente_retencion	es_agente_retencion
3	es_contribuyente_especial	es_contribuyente_especial
4	estado_activo	estado_activo
5	antiguedad_anos	antiguedad_anos
6	sector_ciiu_macro_C	log_empleados
7	sector_ciiu_macro_G	log_ingresos
8	sector_ciiu_macro_K	log_activos
9	sector_ciiu_macro_M	segmento
10	sector_ciiu_macro_OTRO	liquidez_corriente
11	sector_ciiu_macro_S	margen_operacional
12	region_Guayas	sector_ciiu_macro_C
13	region_Pichincha	sector_ciiu_macro_G
14	region_Resto	sector_ciiu_macro_K
15	NaN	sector_ciiu_macro_M
16	NaN	sector_ciiu_macro_OTRO
17	NaN	sector_ciiu_macro_S
18	NaN	region_Guayas
19	NaN	region_Pichincha
20	NaN	region_Resto

```
[4]: # Corrección: deduplicar Capa 1 por RUC antes de modelar
# Causa: la misma empresa aparece en la fuente HORAS y también en LEADS.
# Las features SRI son idénticas (mismo RUC → mismo registro SRI).
# Regla: si alguna fila del RUC tiene es_cliente_fpa=1, el RUC se marca como 1.

dups_ruc = labels_c1[labels_c1["RUC"].duplicated(keep=False)][
    ["name_norm", "RUC", "source_label", "es_cliente_fpa"]
].sort_values("RUC")
print(f"RUCs duplicados antes de corregir: {dups_ruc['RUC'].nunique()} únicos,
      ↪ en {len(dups_ruc)} filas")
print(dups_ruc.to_string(index=False))
print()

# Ordenar: primero los que son clientes FPA (=1), luego los que no (=0)
# → drop_duplicates 'first' conserva el de mayor es_cliente_fpa
idx_ok = (
    labels_c1
```

```

.sort_values("es_cliente_fpa", ascending=False)
.drop_duplicates(subset="RUC", keep="first")
.index
)

features_c1 = features_c1.loc[idx_ok].reset_index(drop=True)
matriz_c1    = matriz_c1.loc[idx_ok].reset_index(drop=True)
labels_c1    = labels_c1.loc[idx_ok].reset_index(drop=True)

print(f"Capa 1 corregida: {len(features_c1)} empresas | {labels_c1['RUC'].
↪nunique()} RUCs únicos")
print(f"Clientes FPA: {int(labels_c1['es_cliente_fpa'].sum())}")
assert labels_c1["RUC"].duplicated().sum() == 0
assert len(features_c1) == len(matriz_c1) == len(labels_c1)

```

RUCs duplicados antes de corregir: 5 únicos en 15 filas

	name_norm	RUC	source_label	es_cliente_fpa
	GNP SEGUROS	0106644651001	LEADS	0
	PRIMERO SEGUROS	0106644651001	LEADS	0
	MTWA SOLUCIONES INTEGRALES	0190351084001	LEADS	0
	SOLUCIONES INTEGRALES IKNELIA	0190351084001	LEADS	0
	GRUPO CARSO	0913220786001	LEADS	0
	GRUPO SID	0913220786001	LEADS	0
	GRUPO TMM	0913220786001	LEADS	0
	GRUPO TRIMEX	0913220786001	LEADS	0
	GRUPO VASCONIA SAB	0913220786001	LEADS	0
	GRUPO ZUCARMEX	0913220786001	LEADS	0
	MILLICOM TELEFONICA PA NI	1710256999001	HORAS	1
	TELEFONICA CR	1710256999001	HORAS	1
	TELEFONICA EC	1710256999001	HORAS	1
	CLAYTON MEXICO C V	1792424585001	LEADS	0
	NEOLPHARMA C V	1792424585001	LEADS	0

Capa 1 corregida: 157 empresas | 157 RUCs únicos

Clientes FPA: 51

1.2 2. Funciones auxiliares

Se definen utilidades para:

1. visualizar el dendrograma de Ward,
2. evaluar candidatos de K con K-Means y clustering jerarquico,
3. construir perfiles de negocio por cluster,
4. exportar artefactos finales.

```

[5]: def plot_ward_dendrogram(X, title, figsize=(14, 5)):
      Z = linkage(X, method="ward")
      plt.figure(figsize=figsize)

```

```

dendrogram(Z, no_labels=True, color_threshold=None)
plt.title(title)
plt.xlabel("Observaciones")
plt.ylabel("Distancia Ward")
plt.tight_layout()
plt.show()
return Z

def evaluate_kmeans_grid(X, k_values, random_state=RANDOM_STATE, n_init=25):
    rows = []
    for k in k_values:
        model = KMeans(n_clusters=k, random_state=random_state, n_init=n_init)
        labels = model.fit_predict(X)
        counts = pd.Series(labels).value_counts()
        rows.append(
            {
                "k": k,
                "inertia": model.inertia_,
                "silhouette": silhouette_score(X, labels),
                "davies_bouldin": davies_bouldin_score(X, labels),
                "min_cluster_size": int(counts.min()),
                "max_cluster_size": int(counts.max()),
            }
        )
    return pd.DataFrame(rows)

def evaluate_ward_grid(X, k_values):
    rows = []
    for k in k_values:
        labels = AgglomerativeClustering(n_clusters=k, linkage="ward").
        ↪fit_predict(X)
        counts = pd.Series(labels).value_counts()
        rows.append(
            {
                "k": k,
                "silhouette": silhouette_score(X, labels),
                "davies_bouldin": davies_bouldin_score(X, labels),
                "min_cluster_size": int(counts.min()),
                "max_cluster_size": int(counts.max()),
            }
        )
    return pd.DataFrame(rows)

def plot_kmeans_diagnostics(metrics_df, title, chosen_k=None):

```

```

fig, axes = plt.subplots(1, 3, figsize=(18, 4))

sns.lineplot(data=metrics_df, x="k", y="inertia", marker="o", ax=axes[0],
color="#1f77b4")
axes[0].set_title(f"{title} - Elbow")
axes[0].set_ylabel("Inertia (WCSS)")

sns.lineplot(data=metrics_df, x="k", y="silhouette", marker="o",
ax=axes[1], color="#2ca02c")
axes[1].set_title(f"{title} - Silhouette")
axes[1].set_ylabel("Silhouette")

sns.lineplot(data=metrics_df, x="k", y="davies_bouldin", marker="o",
ax=axes[2], color="#d62728")
axes[2].set_title(f"{title} - Davies-Bouldin")
axes[2].set_ylabel("Davies-Bouldin")

if chosen_k is not None:
    for ax in axes:
        ax.axvline(chosen_k, color="black", linestyle="--", alpha=0.7)

for ax in axes:
    ax.set_xticks(metrics_df["k"])

plt.tight_layout()
plt.show()

def top_category(series):
    moda = series.mode(dropna=True)
    if len(modas) == 0:
        return np.nan
    return moda.iloc[0]

def summarize_clusters(raw_df, cluster_col, continuous_cols, binary_cols,
categorical_cols):
    summary = raw_df.groupby(cluster_col).size().rename("n").to_frame()
    summary["tasa_clientes_fpa"] = raw_df.
groupby(cluster_col)["es_cliente_fpa"].mean()

    for col in continuous_cols:
        summary[f"mediana_{col}"] = raw_df.groupby(cluster_col)[col].median()

    for col in binary_cols:
        summary[f"prop_{col}"] = raw_df.groupby(cluster_col)[col].mean()

```

```

    for col in categorical_cols:
        summary[f"moda_{col}"] = raw_df.groupby(cluster_col)[col].
        ↪agg(top_category)

    return summary.reset_index()

def plot_centroid_heatmap(centroids_df, title):
    plt.figure(figsize=(16, max(4, 1.1 * len(centroids_df))))
    sns.heatmap(centroids_df, cmap="coolwarm", center=0, annot=True, fmt=".2f")
    plt.title(title)
    plt.xlabel("Features escaladas")
    plt.ylabel("Clusters")
    plt.tight_layout()
    plt.show()

def plot_cluster_validation(summary_df, cluster_col, title):
    fig, axes = plt.subplots(1, 2, figsize=(14, 4))

    sns.barplot(data=summary_df, x=cluster_col, y="n", ax=axes[0],
    ↪color="#4C78A8")
    axes[0].set_title(f"{title} - Tamaño")
    axes[0].set_ylabel("Empresas")

    sns.barplot(data=summary_df, x=cluster_col, y="tasa_clientes_fpa",
    ↪ax=axes[1], color="#F58518")
    axes[1].set_title(f"{title} - Tasa clientes FPA")
    axes[1].set_ylabel("Proporcion")
    axes[1].set_ylim(0, max(0.5, summary_df["tasa_clientes_fpa"].max() + 0.05))

    plt.tight_layout()
    plt.show()

```

1.3 3. Exploracion jerarquica inicial

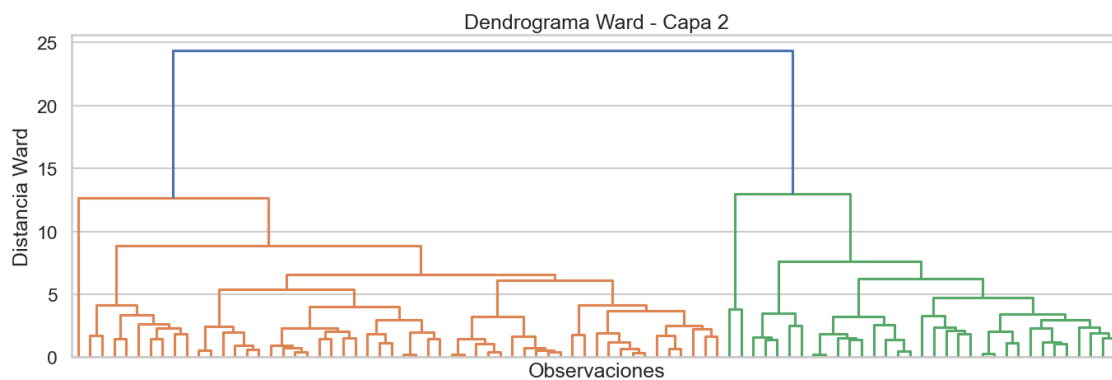
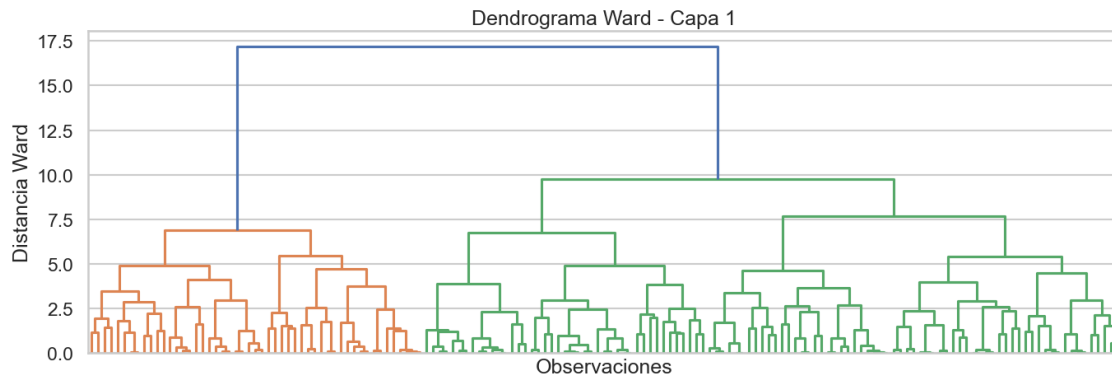
Ward se usa como herramienta exploratoria para identificar cortes plausibles antes de fijar K con K-Means.

```

[6]: Z_c1 = plot_ward_dendrogram(matriz_c1, "Dendrograma Ward - Capa 1")
    Z_c2 = plot_ward_dendrogram(matriz_c2, "Dendrograma Ward - Capa 2")

    print("Ultimas 10 distancias de fusion - Capa 1:", np.round(Z_c1[-10:, 2], 3))
    print("Ultimas 10 distancias de fusion - Capa 2:", np.round(Z_c2[-10:, 2], 3))

```

Ultimas 10 distancias de fusion - Capa 1: [4.717 4.913 4.92 5.4 5.454
6.761 6.9 7.664 9.753 17.192]

Ultimas 10 distancias de fusion - Capa 2: [4.733 5.418 6.097 6.256 6.601
7.639 8.871 12.666 12.984 24.38]

1.4 4. Selecccion de K

La decision combina:

- Elbow sobre inercia,
- Silhouette,
- Davies-Bouldin,
- tamano minimo de cluster para evitar segmentos espurios.

```
[7]: metricas_kmeans_c1 = evaluate_kmeans_grid(matriz_c1, K_RANGE)
metricas_kmeans_c2 = evaluate_kmeans_grid(matriz_c2, K_RANGE)
metricas_ward_c1 = evaluate_ward_grid(matriz_c1, range(2, 7))
metricas_ward_c2 = evaluate_ward_grid(matriz_c2, range(2, 7))

display(metricas_kmeans_c1.round(3))
display(metricas_ward_c1.round(3))
```

```
display(mettricas_kmeans_c2.round(3))
display(mettricas_ward_c2.round(3))
```

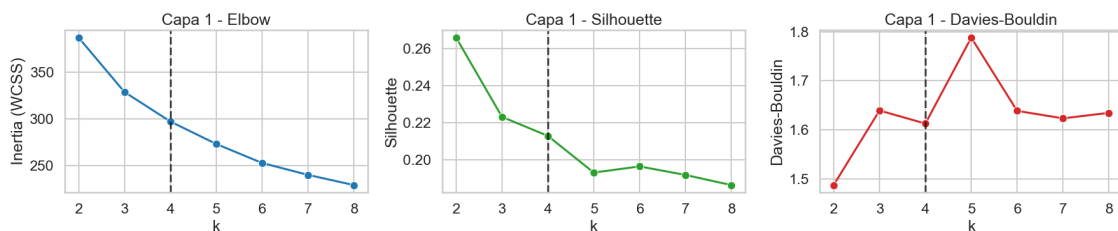
	k	inertia	silhouette	davies_bouldin	min_cluster_size	max_cluster_size
0	2	386.977	0.266	1.487	55	102
1	3	328.741	0.223	1.639	39	68
2	4	297.160	0.213	1.613	34	50
3	5	273.128	0.193	1.788	25	37
4	6	252.785	0.196	1.639	20	35
5	7	240.059	0.192	1.623	18	30
6	8	229.127	0.186	1.635	13	32

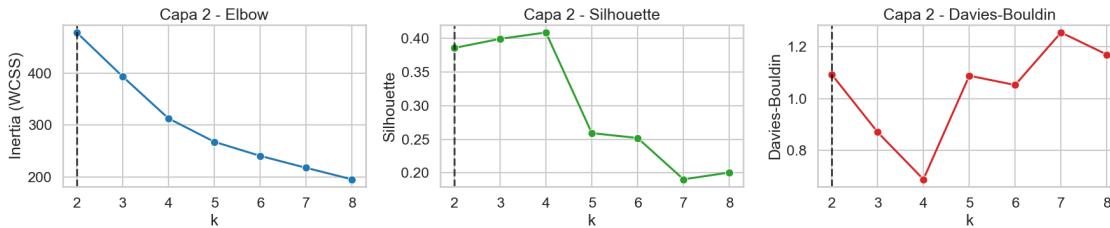
	k	silhouette	davies_bouldin	min_cluster_size	max_cluster_size
0	2	0.257	1.506	51	106
1	3	0.189	1.854	43	63
2	4	0.185	1.731	28	51
3	5	0.162	1.924	24	43
4	6	0.164	1.800	16	35

	k	inertia	silhouette	davies_bouldin	min_cluster_size	max_cluster_size
0	2	477.748	0.386	1.093	35	52
1	3	393.460	0.399	0.870	2	52
2	4	313.026	0.409	0.688	1	50
3	5	268.114	0.259	1.088	1	33
4	6	241.137	0.252	1.052	1	41
5	7	218.272	0.190	1.254	1	23
6	8	196.158	0.200	1.169	1	21

	k	silhouette	davies_bouldin	min_cluster_size	max_cluster_size
0	2	0.385	1.096	33	54
1	3	0.397	0.866	2	54
2	4	0.408	0.681	1	53
3	5	0.257	1.002	1	44
4	6	0.253	1.034	1	44

```
[8]: plot_kmeans_diagnostics(mettricas_kmeans_c1, "Capa 1", chosen_k=4)
plot_kmeans_diagnostics(mettricas_kmeans_c2, "Capa 2", chosen_k=2)
```





```
[9]: K_CAPA1 = 4
      K_CAPA2 = 2
      K_CAPA2_SENSIBILIDAD = 4

      print("Decision final de K")
      print("- Capa 1: K=4 -> mejor equilibrio entre interpretabilidad, tamanos_
        ↳ balanceados y valor de negocio.")
      print("- Capa 2: K=2 -> evita clusters de 1-2 empresas que aparecen desde K>=3_
        ↳ por outliers financieros.")
      print("- Sensibilidad adicional: tambien se corre Capa 2 con K=4 para comparar_
        ↳ contra el mismo K del modelo principal.")
```

Decision final de K

- Capa 1: K=4 -> mejor equilibrio entre interpretabilidad, tamanos balanceados y valor de negocio.
- Capa 2: K=2 -> evita clusters de 1-2 empresas que aparecen desde K>=3 por outliers financieros.
- Sensibilidad adicional: tambien se corre Capa 2 con K=4 para comparar contra el mismo K del modelo principal.

1.5 5. Modelo principal - Capa 1

Se ajusta K-Means con K=4 sobre la matriz SRI, y luego se interpretan los clusters usando las features crudas para no perder semantica de negocio.

```
[10]: modelo_c1 = KMeans(n_clusters=K_CAPA1, random_state=RANDOM_STATE, n_init=25)
      clusters_c1 = modelo_c1.fit_predict(matriz_c1)

      segmentos_c1 = {
          0: "Sociedades jovenes de alta afinidad FPA",
          1: "Grandes corporativos maduros establecidos",
          2: "Empresas comerciales de Guayas",
          3: "Personas naturales del resto del pais",
      }

      asignaciones_c1 = labels_c1.copy()
      asignaciones_c1["cluster_capa1"] = clusters_c1
```

```

asignaciones_c1["segmento_capa1"] = asignaciones_c1["cluster_capa1"].
↳map(segmentos_c1)

perfil_raw_c1 = features_c1.copy()
perfil_raw_c1["cluster_capa1"] = clusters_c1
perfil_raw_c1["segmento_capa1"] = perfil_raw_c1["cluster_capa1"].
↳map(segmentos_c1)

resumen_c1 = summarize_clusters(
    raw_df=perfil_raw_c1,
    cluster_col="cluster_capa1",
    continuous_cols=["antiguedad_anos"],
    binary_cols=[
        "tipo_sociedad",
        "obligado_contabilidad",
        "es_agente_retencion",
        "es_contribuyente_especial",
        "estado_activo",
    ],
    categorical_cols=["sector_ciiu_macro", "region", "source_label"],
)
resumen_c1["segmento_capa1"] = resumen_c1["cluster_capa1"].map(segmentos_c1)
resumen_c1 = resumen_c1[
    [
        "cluster_capa1",
        "segmento_capa1",
        "n",
        "tasa_clientes_fpa",
        "mediana_antiguedad_anos",
        "prop_tipo_sociedad",
        "prop_obligado_contabilidad",
        "prop_es_agente_retencion",
        "prop_es_contribuyente_especial",
        "prop_estado_activo",
        "moda_sector_ciiu_macro",
        "moda_region",
        "moda_source_label",
    ]
].sort_values("cluster_capa1")

display(resumen_c1.round(3))

```

	cluster_capa1	segmento_capa1	n \
0	0	Sociedades jovenes de alta afinidad FPA	50
1	1	Grandes corporativos maduros establecidos	34
2	2	Empresas comerciales de Guayas	38
3	3	Personas naturales del resto del pais	35

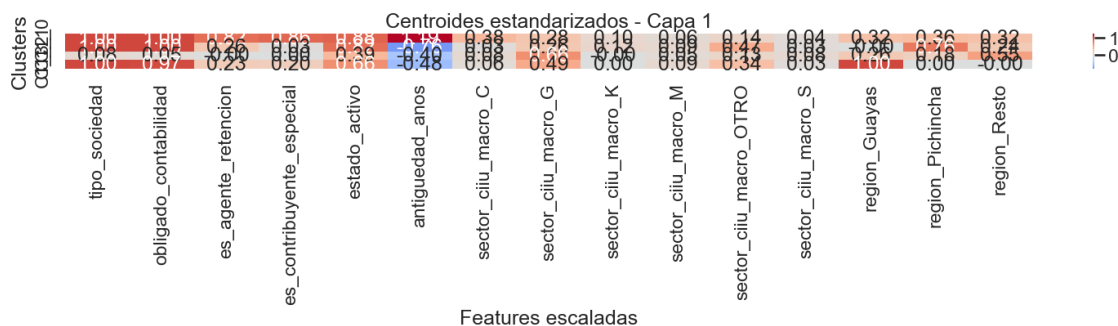
	tasa_clientes_fpa	mediana_antiguedad_anos	prop_tipo_sociedad	\
0	0.480	47.5	1.000	
1	0.500	8.5	1.000	
2	0.105	15.0	0.079	
3	0.171	13.0	1.000	

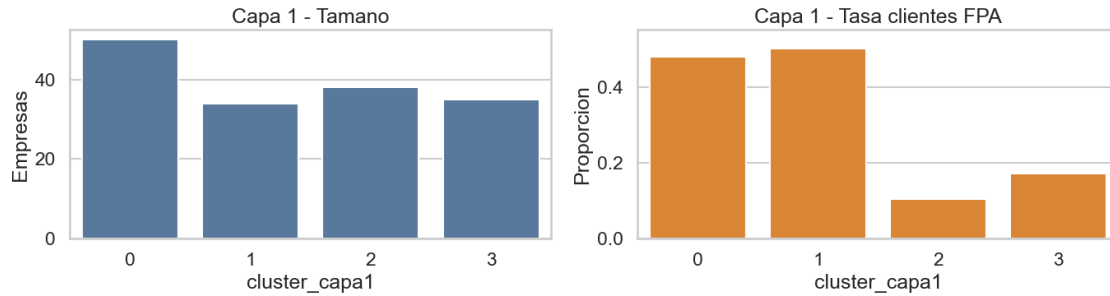
	prop_obligado_contabilidad	prop_es_agente_retencion	\
0	1.000	0.820	
1	1.000	0.265	
2	0.053	0.000	
3	0.971	0.229	

	prop_es_contribuyente_especial	prop_estado_activo	moda_sector_ciiu_macro	\
0	0.860	0.880	C	
1	0.029	0.824	OTRO	
2	0.000	0.395	G	
3	0.200	0.657	G	

	moda_region	moda_source_label
0	Pichincha	LEADS
1	Pichincha	HORAS
2	Resto	LEADS
3	Guayas	LEADS

```
[11]: centroides_c1 = pd.DataFrame(modelo_c1.cluster_centers_, columns=matriz_c1.
    ↪columns)
centroides_c1.index = [f"C1_{cluster}" for cluster in range(K_CAPA1)]
plot_centroid_heatmap(centroides_c1, "Centroides estandarizados - Capa 1")
plot_cluster_validation(resumen_c1, "cluster_capa1", "Capa 1")
```





1.6 6. Modelo de robustez - Capa 2

Primero se ajusta la solución estable ($K=2$). Luego se ejecuta una corrida de sensibilidad con $K=4$ para mostrar cómo los outliers financieros fragmentan artificialmente la muestra.

```
[12]: modelo_c2 = KMeans(n_clusters=K_CAPA2, random_state=RANDOM_STATE, n_init=25)
clusters_c2 = modelo_c2.fit_predict(matriz_c2)

segmentos_c2 = {
    0: "Empresas pequeñas y recientes",
    1: "Empresas medianas-grandes consolidadas",
}

asignaciones_c2 = labels_c2.copy()
asignaciones_c2["cluster_capa2"] = clusters_c2
asignaciones_c2["segmento_capa2"] = asignaciones_c2["cluster_capa2"].
    ↪map(segmentos_c2)

perfil_raw_c2 = features_c2.copy()
perfil_raw_c2["cluster_capa2"] = clusters_c2
perfil_raw_c2["segmento_capa2"] = perfil_raw_c2["cluster_capa2"].
    ↪map(segmentos_c2)

resumen_c2 = summarize_clusters(
    raw_df=perfil_raw_c2,
    cluster_col="cluster_capa2",
    continuous_cols=[
        "antiguedad_anos",
        "log_empleados",
        "log_ingresos",
        "log_activos",
        "segmento",
        "liquidez_corriente",
        "margen_operacional",
    ],
    binary_cols=[
```

```

        "tipo_sociedad",
        "obligado_contabilidad",
        "es_agente_retencion",
        "es_contribuyente_especial",
        "estado_activo",
    ],
    categorical_cols=["sector_ciiu_macro", "region", "source_label"],
)
resumen_c2["segmento_capa2"] = resumen_c2["cluster_capa2"].map(segmentos_c2)
resumen_c2 = resumen_c2.sort_values("cluster_capa2")

display(resumen_c2.round(3))

```

	cluster_capa2	n	tasa_clientes_fpa	mediana_antiguedad_anos	\
0	0	35	0.429	8.0	
1	1	52	0.423	36.5	

	mediana_log_empleados	mediana_log_ingresos	mediana_log_activos	\
0	0.602	0.000	4.155	
1	2.293	7.513	7.504	

	mediana_segmento	mediana_liquidez_corriente	mediana_margen_operacional	\
0	1.0	0.84	0.0	
1	4.0	1.57	0.0	

	prop_tipo_sociedad	prop_obligado_contabilidad	prop_es_agente_retencion	\
0	1.0	0.971	0.200	
1	1.0	1.000	0.846	

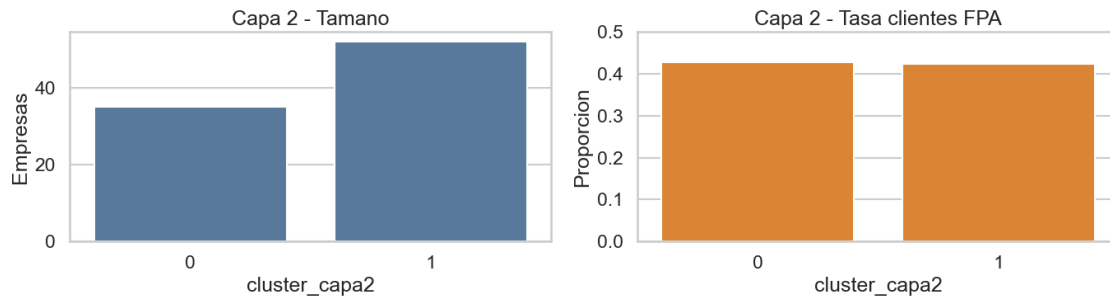
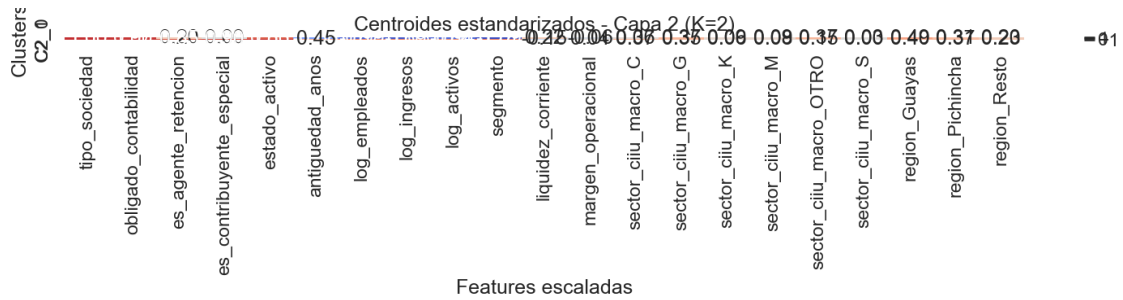
	prop_es_contribuyente_especial	prop_estado_activo	moda_sector_ciiu_macro	\
0	0.000	0.829	G	
1	0.904	1.000	C	

	moda_region	moda_source_label	segmento_capa2
0	Guayas	LEADS	Empresas pequenas y recientes
1	Guayas	LEADS	Empresas medianas-grandes consolidadas

```

[13]: centroides_c2 = pd.DataFrame(modelo_c2.cluster_centers_, columns=matriz_c2.
    ↪columns)
centroides_c2.index = [f"C2_{cluster}" for cluster in range(K_CAPA2)]
plot_centroid_heatmap(centroides_c2, "Centroides estandarizados - Capa 2 (K=2)")
plot_cluster_validation(resumen_c2, "cluster_capa2", "Capa 2")

```



```
[14]: modelo_c2_k4 = KMeans(n_clusters=K_CAPA2_SENSIBILIDAD,
    random_state=RANDOM_STATE, n_init=25)
clusters_c2_k4 = modelo_c2_k4.fit_predict(matriz_c2)

sensibilidad_c2 = features_c2.copy()
sensibilidad_c2["cluster_capa2_k4"] = clusters_c2_k4

resumen_c2_k4 = sensibilidad_c2.groupby("cluster_capa2_k4").agg(
    n=("RUC", "size"),
    tasa_clientes_fpa=("es_cliente_fpa", "mean"),
    mediana_antiguedad_anos=("antiguedad_anos", "median"),
    mediana_log_ingresos=("log_ingresos", "median"),
    mediana_liquidez_corriente=("liquidez_corriente", "median"),
    mediana_margen_operacional=("margen_operacional", "median"),
).reset_index()

clusters_pequenos = (
    sensibilidad_c2["cluster_capa2_k4"].value_counts().loc[lambdas: s <= 2].
    index.tolist()
)

outliers_c2_k4 = sensibilidad_c2.loc[
    sensibilidad_c2["cluster_capa2_k4"].isin(clusters_pequenos),
    [
        "cluster_capa2_k4",
```



```

        "name_norm",
        "RUC",
        "es_cliente_fpa",
        "sector_ciiu_macro",
        "region",
        "anio",
        "log_empleados",
        "log_ingresos",
        "log_activos",
        "segmento",
        "liquidez_corriente",
        "margen_operacional",
    ],
].sort_values(["cluster_capa2_k4", "name_norm"])

display(resumen_c2_k4.round(3))
display(outliers_c2_k4)

```

	cluster_capa2_k4	n	tasa_clientes_fpa	mediana_antiguedad_anos	\
0	0	34	0.412	8.0	
1	1	50	0.440	36.5	
2	2	2	0.500	9.5	
3	3	1	0.000	30.0	

	mediana_log_ingresos	mediana_liquidez_corriente	\
0	0.00	0.840	
1	7.54	1.520	
2	2.36	3408.466	
3	7.37	1.680	

	mediana_margen_operacional
0	0.000
1	0.000
2	0.000
3	6316.733

	cluster_capa2_k4	name_norm	RUC	es_cliente_fpa	\
33	2	FRESENIUS MEDICAL CARE	1792333881001	0	
86	2	XCARET	2490398352001	1	
12	3	COCA COLA	1791324404001	0	

	sector_ciiu_macro	region	anio	log_empleados	log_ingresos	\
33	K	Pichincha	2024.0	0.477121	0.000000	
86	G	Resto	2025.0	0.477121	4.719105	
12	M	Pichincha	2025.0	1.447158	7.369719	

	log_activos	segmento	liquidez_corriente	margen_operacional
33	7.129420	1.0	2723.0900	0.0000

86	4.810868	1.0	4093.8412	0.0000
12	7.188918	4.0	1.6800	6316.7326

1.7 7. Sesgo de cobertura entre capas

Una pregunta clave para la tesis es si el subconjunto financiero de Capa 2 representa por igual a todos los perfiles descubiertos en Capa 1.

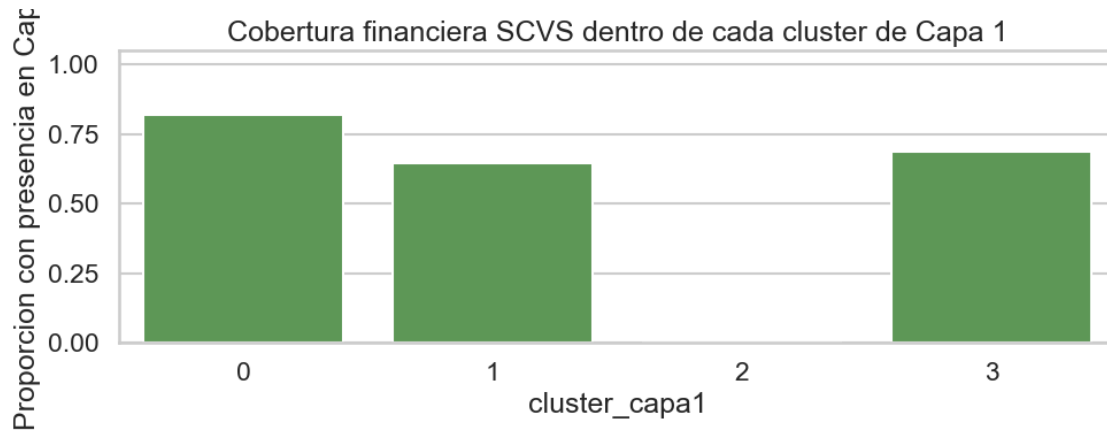
```
[15]: cobertura_c2 = asignaciones_c1.copy()
cobertura_c2["en_capa2"] = cobertura_c2["RUC"].isin(set(asignaciones_c2["RUC"]))
cobertura_c2 = cobertura_c2.groupby(["cluster_capa1", "segmento_capa1"]).agg(
    n_total=("RUC", "size"),
    n_con_financieras=("en_capa2", "sum"),
    cobertura_financiera=("en_capa2", "mean"),
    tasa_clientes_fpa=("es_cliente_fpa", "mean"),
).reset_index().sort_values("cluster_capa1")

display(cobertura_c2.round(3))

plt.figure(figsize=(10, 4))
sns.barplot(data=cobertura_c2, x="cluster_capa1", y="cobertura_financiera",
            color="#54A24B")
plt.title("Cobertura financiera SCVS dentro de cada cluster de Capa 1")
plt.ylabel("Proporcion con presencia en Capa 2")
plt.ylim(0, 1.05)
plt.tight_layout()
plt.show()
```

cluster_capa1	segmento_capa1	n_total	\
0	0	Sociedades jovenes de alta afinidad FPA	50
1	1	Grandes corporativos maduros establecidos	34
2	2	Empresas comerciales de Guayas	38
3	3	Personas naturales del resto del pais	35

n_con_financieras	cobertura_financiera	tasa_clientes_fpa
0	41	0.820
1	22	0.647
2	0	0.000
3	24	0.686



1.8 8. Comparacion de robustez entre capas

Se comparan las empresas comunes por RUC usando:

- ARI con el mejor K de cada capa (4 vs 2),
- ARI bajo la misma granularidad (4 vs 4) como sensibilidad adicional.

```
[16]: comparacion_best = asignaciones_c2.merge(
    asignaciones_c1[["RUC", "cluster_capa1", "segmento_capa1"]],
    on="RUC",
    how="left",
)

asignaciones_c2_k4 = labels_c2.copy()
asignaciones_c2_k4["cluster_capa2_k4"] = clusters_c2_k4

comparacion_same_k = asignaciones_c2_k4.merge(
    asignaciones_c1[["RUC", "cluster_capa1", "segmento_capa1"]],
    on="RUC",
    how="left",
)

ari_best = adjusted_rand_score(comparacion_best["cluster_capa1"],
    comparacion_best["cluster_capa2"])
ari_same_k = adjusted_rand_score(
    comparacion_same_k["cluster_capa1"],
    comparacion_same_k["cluster_capa2_k4"],
)

print(f"ARI mejor K por capa (Capa1=4 vs Capa2=2): {ari_best:.3f}")
print(f"ARI misma granularidad (Capa1=4 vs Capa2=4): {ari_same_k:.3f}")

tabla_cruce_best = pd.crosstab(
```

```

    comparacion_best["cluster_capa1"],
    comparacion_best["cluster_capa2"],
)
tabla_cruce_same_k = pd.crosstab(
    comparacion_same_k["cluster_capa1"],
    comparacion_same_k["cluster_capa2_k4"],
)

display(tabla_cruce_best)
display(tabla_cruce_same_k)

```

ARI mejor K por capa (Capa1=4 vs Capa2=2): 0.376
ARI misma granularidad (Capa1=4 vs Capa2=4): 0.324

```

cluster_capa2    0    1
cluster_capa1
0                2   39
1               17    5
3               16    8

```

```

cluster_capa2_k4    0    1    2    3
cluster_capa1
0                 3   37    0    1
1                15    5    2    0
3                16    8    0    0

```

```

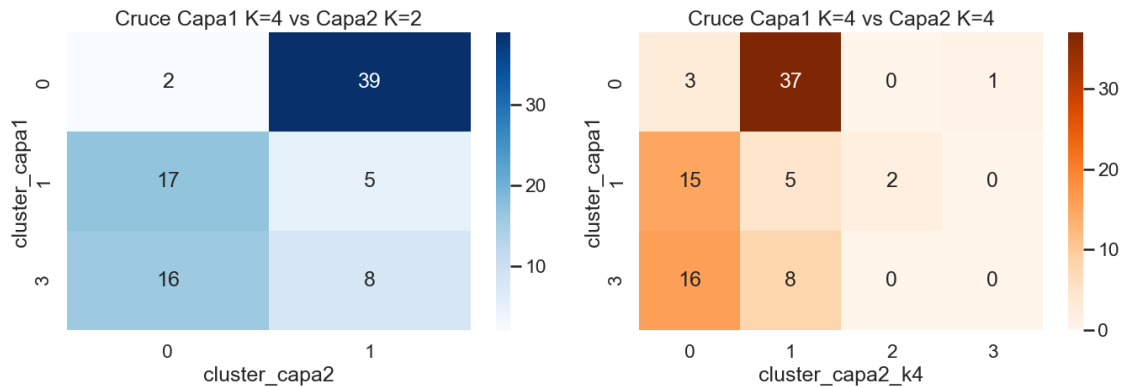
[17]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.heatmap(tabla_cruce_best, annot=True, fmt="d", cmap="Blues", ax=axes[0])
axes[0].set_title("Cruce Capa1 K=4 vs Capa2 K=2")
axes[0].set_xlabel("cluster_capa2")
axes[0].set_ylabel("cluster_capa1")

sns.heatmap(tabla_cruce_same_k, annot=True, fmt="d", cmap="Oranges", ax=axes[1])
axes[1].set_title("Cruce Capa1 K=4 vs Capa2 K=4")
axes[1].set_xlabel("cluster_capa2_k4")
axes[1].set_ylabel("cluster_capa1")

plt.tight_layout()
plt.show()

```



1.9 9. Exportacion de resultados

Se guardan las metricas, las etiquetas de cluster y los resuenes de perfil para ser reutilizados en evaluacion, reporting o scoring posterior.

```
[18]: metricas_kmeans_c1.to_csv(PATH_OUT / "metricas_kmeans_capa1.csv", index=False)
metricas_kmeans_c2.to_csv(PATH_OUT / "metricas_kmeans_capa2.csv", index=False)
metricas_ward_c1.to_csv(PATH_OUT / "metricas_ward_capa1.csv", index=False)
metricas_ward_c2.to_csv(PATH_OUT / "metricas_ward_capa2.csv", index=False)

asignaciones_c1.to_csv(PATH_OUT / "clusters_capa1.csv", index=False)
asignaciones_c2.to_csv(PATH_OUT / "clusters_capa2.csv", index=False)
asignaciones_c2_k4.to_csv(PATH_OUT / "clusters_capa2_k4_sensibilidad.csv",
    ↪ index=False)

resumen_c1.to_csv(PATH_OUT / "perfil_clusters_capa1.csv", index=False)
resumen_c2.to_csv(PATH_OUT / "perfil_clusters_capa2.csv", index=False)
resumen_c2_k4.to_csv(PATH_OUT / "perfil_clusters_capa2_k4.csv", index=False)
cobertura_c2.to_csv(PATH_OUT / "cobertura_financiera_por_cluster_capa1.csv",
    ↪ index=False)
comparacion_best.to_csv(PATH_OUT / "comparacion_capas_best_k.csv", index=False)
comparacion_same_k.to_csv(PATH_OUT / "comparacion_capas_same_k.csv",
    ↪ index=False)

archivos_generados = sorted([path.name for path in PATH_OUT.glob("*.csv")])
pd.DataFrame({"archivo_generado": archivos_generados})
```

```
[18]:          archivo_generado
0          clusters_capa1.csv
1          clusters_capa2.csv
2  clusters_capa2_k4_sensibilidad.csv
3  cobertura_financiera_por_cluster_capa1.csv
4          comparacion_capas_best_k.csv
```

```
5             comparacion_capas_same_k.csv
6             metricas_kmeans_capa1.csv
7             metricas_kmeans_capa2.csv
8             metricas_ward_capa1.csv
9             metricas_ward_capa2.csv
10            perfil_clusters_capa1.csv
11            perfil_clusters_capa2.csv
12            perfil_clusters_capa2_k4.csv
```

1.10 10. Conclusiones ejecutivas

1. **Capa 1** con $K=4$ entrega una segmentacion interpretable y balanceada para el modelo principal de la tesis.
2. **Capa 2** con $K=2$ es la solucion estable cuando se incorporan variables financieras; con $K \geq 3$ aparecen clusters diminutos inducidos por outliers.
3. La concordancia entre capas es parcial (ARI moderado-bajo), lo que sugiere que la firma financiera agrega estructura nueva y no solo refina la segmentacion SRI.
4. El sesgo de cobertura es material: el cluster menos formalizado de **Capa 1** no tiene presencia financiera en **Capa 2**, por lo que el enriquecimiento SCVS no representa por igual a todos los perfiles del universo comercial.